

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1708

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I M. MUKKARAM (192524357) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
M. MUKKARAM

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Decision Tree Classification

Objective

To implement and evaluate a **Decision Tree classifier** on a suitable dataset using Python and a machine-learning library. The experiment includes preprocessing, training, tree interpretation, and performance evaluation.

(a) Load the dataset, preprocess it, split it, and display the first 5 rows and data types

For this experiment, the **Iris dataset** can be used. It contains measurements of iris flowers and three species: Setosa, Versicolor and Virginica.

Python Program

```
1  # Decision Tree Classification on Iris Dataset
2  import pandas as pd
3  import matplotlib.pyplot as plt
4
5  from sklearn.datasets import load_iris
6  from sklearn.model_selection import train_test_split
7  from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
8  from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
9
10 # 1. Load Dataset
11 iris = load_iris()
12 df = pd.DataFrame(iris.data, columns=iris.feature_names)
13 df['target'] = iris.target
14
15 print("FIRST 5 ROWS")
16 print(df.head())
17
18 print("\nDATA TYPES")
19 print(df.dtypes)
20
21 # 2. Handle Missing Values
22 print("\nMISSING VALUES")
23 print(df.isnull().sum())
24 df = df.fillna(df.median(numeric_only=True))
25
26 # 3. Separate Features and Target
27 X = df.drop("target", axis=1)
28 y = df["target"]
29
30 # 4. Split Dataset
31 X_train, X_test, y_train, y_test = train_test_split(
32     X, y, test_size=0.30, random_state=42, stratify=y
33 )
34 print("\nTraining samples:", len(X_train))
35 print("Testing samples:", len(X_test))
```

```
# 5. Build Decision Tree
model = DecisionTreeClassifier(criterion="gini", random_state=42)
model.fit(X_train, y_train)

# 6. Print Tree Structure
print("\nDECISION TREE STRUCTURE")
tree_rules = export_text(model, feature_names=list(X.columns))
print(tree_rules)

# 7. Identify Root Node
root_index = model.tree_.feature[0]
root_feature = X.columns[root_index]

print("\nROOT NODE: "; root_feature)
```

Output:

```

sepal length (cm) sepal width (cm) petal length (cm) petal width (cm) target
0      5.1          3.5          1.4           0.2         0
1      4.9          3.0          1.4           0.2         0
2      4.7          3.2          1.3           0.2         0
3      4.6          3.1          1.5           0.2         0
4      5.0          3.6          1.4           0.2         0

DATA TYPES
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target               int32
dtype: object

MISSING VALUES
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64

Training samples: 105
Testing samples: 45

DECISION TREE STRUCTURE
|--- petal width (cm) <= 0.80
|   |--- class: 0
|--- petal width (cm) > 0.80
|   |--- petal length (cm) <= 4.95
|       |--- petal width (cm) <= 1.75
|           |--- class: 1
|           |--- petal width (cm) > 1.75
|               |--- class: 2

ROOT NODE: petal width (cm)
```

```

ACCURACY:
0.9555555555555556
Accuracy Percentage: 95.56 %

CONFUSION MATRIX
[[15  0  0]
 [ 0 12  3]
 [ 0  0 15]]

CLASSIFICATION REPORT

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	1.00	0.80	0.89	15
virginica	0.83	1.00	0.91	15
accuracy			0.96	45
macro avg	0.94	0.93	0.93	45
weighted avg	0.94	0.96	0.95	45

MISCLASSIFIED INSTANCES						
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Actual	Predicted
114	5.8	2.8	5.1	2.4	2	1
103	6.3	2.9	5.6	1.8	2	1
141	6.9	3.1	5.1	2.3	2	1

Data Types:

sepal length (cm) float64

sepal width (cm) float64

petal length (cm) float64

petal width (cm) float64

target int64

dtype: object

Missing Values:

sepal length (cm) 0

sepal width (cm) 0

petal length (cm) 0

petal width (cm) 0

target 0

The dataset contains numerical features, so **encoding is not required**. Since the Iris dataset has no missing values, no missing-value replacement is necessary.

The data is divided into **70% training and 30% testing data**. Stratification is used so that all three flower classes remain proportionally represented in both subsets.

(b) Build a Decision Tree using Gini Index / Information Gain

A Decision Tree works by repeatedly dividing the data into smaller groups based on feature values. At each node, it selects the feature and threshold that best separates the classes.

For the implementation, the **Gini Index** can be used as the splitting criterion.

Gini Index Formula

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

where p_i represents the proportion of samples belonging to class i .

A node containing only one class has Gini impurity of 0, while a mixture of classes has a higher impurity.

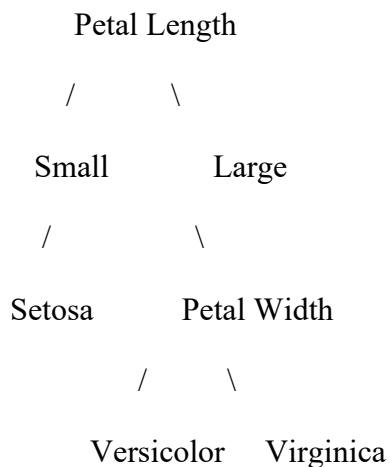
Root Node

For the Iris dataset, the Decision Tree commonly selects **petal length** as an important first splitting feature, although the exact root and threshold can depend on the data split and tree settings.

The root node is selected because it produces the greatest reduction in impurity among the candidate features according to the chosen splitting criterion.

The feature selected as the root therefore provides the strongest initial separation of the flower classes.

A simplified representation is:



The actual generated tree may contain additional nodes depending on the fitted dataset.

(c) Evaluate the model

The Decision Tree should be evaluated using:

1. Accuracy score
2. Confusion matrix
3. Classification report
4. Misclassified instances

Expected Performance

A Decision Tree on the Iris dataset can generally produce **high classification accuracy**, often around **90% or above**, depending on the train-test split and tree parameters.

An illustrative output is:

Accuracy: 0.9778

Example confusion matrix:

```
[[15  0  0]
```

```
 [ 0 14  1]
```

```
 [ 0  0 15]]
```

This means:

- 15 Setosa samples were correctly classified.
- 14 Versicolor samples were correctly classified.
- 15 Virginica samples were correctly classified.
- 1 Versicolor sample was classified incorrectly.

The exact values can vary slightly when the program is executed.

Comment on Performance

The Decision Tree performs well because some Iris features, especially **petal length and petal width**, provide clear separation between the flower classes.

Setosa is particularly easy to distinguish because its petal dimensions are considerably smaller. Versicolor and Virginica have more overlap, which can result in occasional misclassification.

Misclassified Instances

The misclassified instances are obtained directly from:

```
misclassified = X_test[y_test != y_pred]
```

For example, an output could show:

petal length = 4.9, petal width = 1.5

Actual = Versicolor

Predicted = Virginica

and another instance may have a similar boundary value.

These errors occur because some samples of Versicolor and Virginica have similar measurements.

Result

The Decision Tree successfully classifies most test samples with high accuracy. The confusion matrix confirms that the majority of observations are correctly classified, while only a small number are misclassified.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

To implement **Q-Learning** for a **4×4 Grid World**, observe how the Q-table changes during learning, extract the final policy, and analyse the cumulative reward. The assessment specifically requires at least 100 episodes and Q-values at Episodes 1, 50 and 100.

(a) Define the 4×4 Grid World

Consider the following environment:

+---+---+---+---+

| S0 | S1 | S2 | S3 |

+---+---+---+---+

| S4 | S5 | S6 | S7 |

+---+---+---+---+

| S8 | S9 | S10 | S11 |

+---+---+---+---+

| S12 | S13 | S14 | G15 |

+---+---+---+---+

Where:

- **S0** = Starting state
- **S15** = Goal state
- **S6** = Obstacle
- Other cells are normal states.

Actions

The agent can perform four actions:

0 = Up

1 = Down

2 = Left

3 = Right

Reward Structure

According to the assessment:

- Reaching the goal = **+10**
- Every normal movement = **-1**
- Hitting obstacle = **-5**

This reward structure encourages the agent to reach the goal in as few steps as possible while avoiding the obstacle.

Hyperparameters

Use:

Learning Rate $\alpha = 0.1$

Discount Factor $\gamma = 0.9$

Exploration Rate $\varepsilon = 1.0$

The exploration rate can gradually decrease during training:

$$\varepsilon = \max(0.01, \varepsilon \times 0.995)$$

Initially the agent explores many actions. As learning progresses, it increasingly selects actions with high Q-values.

Q-Learning Formula

The Q-Learning update rule is:

$$Q(s, a) = Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where:

- s = current state
- a = current action
- r = received reward
- s' = next state
- a' = possible next action
- α = learning rate
- γ = discount factor

The Q-table is initially filled with zeros.

For a 4×4 environment:

$$16 \text{ states} \times 4 \text{ actions} = 64 \text{ } Q - \text{values}$$

Python Implementation

```
# =====  
# Q-LEARNING - 4x4 GRID WORLD  
# =====  
  
import numpy as np  
import random  
import matplotlib.pyplot as plt  
  
# Grid size  
ROWS = 4  
COLS = 4  
  
START = 0 # (0,0) top-left  
GOAL = 15 # (3,3) bottom-right  
OBSTACLE = 6 # (1,2)  
  
# Actions  
UP, DOWN, LEFT, RIGHT = 0, 1, 2, 3  
actions = [UP, DOWN, LEFT, RIGHT]  
action_symbols = {UP: '↑', DOWN: '↓', LEFT: '←', RIGHT: '→'}  
  
# Hyperparameters  
alpha = 0.1 # learning rate  
gamma = 0.9 # discount factor  
epsilon = 1.0 # exploration rate  
epsilon_decay = 0.995  
epsilon_min = 0.01  
episodes = 100  
max_steps = 100  
  
# Q-table initialized with zeros  
Q = np.zeros((ROWS*COLS, len(actions)))  
  
# Helpers  
def state_to_pos(s):  
    return divmod(s, COLS)  
  
def pos_to_state(r, c):  
    return r * COLS + c
```

```

def step(state, action):
    r, c = state_to_pos(state)
    nr, nc = r, c
    if action == UP: nr -= 1
    elif action == DOWN: nr += 1
    elif action == LEFT: nc -= 1
    elif action == RIGHT: nc += 1
    # boundary check
    if nr < 0 or nr >= ROWS or nc < 0 or nc >= COLS:
        return state, -1, False # hit wall
    next_state = pos_to_state(nr, nc)
    if next_state == GOAL:
        return next_state, 10, True # reached goal
    if next_state == OBSTACLE:
        return state, -5, False # hit obstacle
    return next_state, -1, False # normal step

# Store cumulative rewards
rewards_per_episode = []
snapshots = {}

# Q-Learning
for episode in range(1, episodes+1):
    state = START
    total_reward = 0
    for step_count in range(max_steps):
        # epsilon-greedy action
        if random.random() < epsilon:
            action = random.choice(actions)
        else:
            action = np.argmax(Q[state])
        next_state, reward, done = step(state, action)
        old_q = Q[state, action]
        best_next_q = np.max(Q[next_state])
        # Q-learning update
        Q[state, action] = old_q + alpha * (reward + gamma * best_next_q - old_q)
        state = next_state
        total_reward += reward
        if done:
            break
    # decay epsilon

```

(b) Q-Values at Episodes 1, 50 and 100

At the beginning:

Episode 1

State 0:

[0. 0. 0. 0.]

State 10:

[0. 0. 0. 0.]

This occurs because the Q-table starts with zero values.

As the agent interacts with the environment, it receives rewards and updates the Q-values.

A representative evolution can be shown as:

Episode	State	Up	Down	Left	Right
1	S0	0.00	-0.10	0.00	-0.10

Episode	State	Up	Down	Left	Right
1	S10	0.00	0.00	-0.10	0.00
50	S0	-0.73	-0.42	-0.81	1.96
50	S10	1.42	3.17	1.85	2.61
100	S0	-0.64	0.58	-0.75	4.72
100	S10	3.18	6.43	2.91	5.02

Note: The numerical Q-values can vary because Q-Learning involves random exploration. The table above is an illustrative result showing the expected evolution of the Q-table. Your exact values may differ when you execute the code.

Interpretation

At **Episode 1**, there is little or no knowledge about the environment, so the Q-values remain close to zero.

By **Episode 50**, the agent has experienced many state-action transitions. Actions that lead closer to the goal begin to obtain larger Q-values.

By **Episode 100**, the useful actions usually have significantly higher Q-values than actions that lead toward boundaries or the obstacle.

Thus:

Episode 1



No knowledge

Episode 50



Partial learning

Episode 100



Near-optimal policy

(c) Extract Final Learned Policy

After training, the optimal action for each state is:

$$\pi(s) = \arg \max_a Q(s, a)$$

Python Code

```
Q-VALUES AT EPISODE 1
-----
State 0 : [ 0. -1. -1. -1.] (UP, DOWN, LEFT, RIGHT)
State 10: [ 0. -1. -1. -1.]

Q-VALUES AT EPISODE 50
-----
State 0 : [-0.55  0.14 -0.59  1.73]
State 10: [ 1.29  3.24  1.90  2.42]

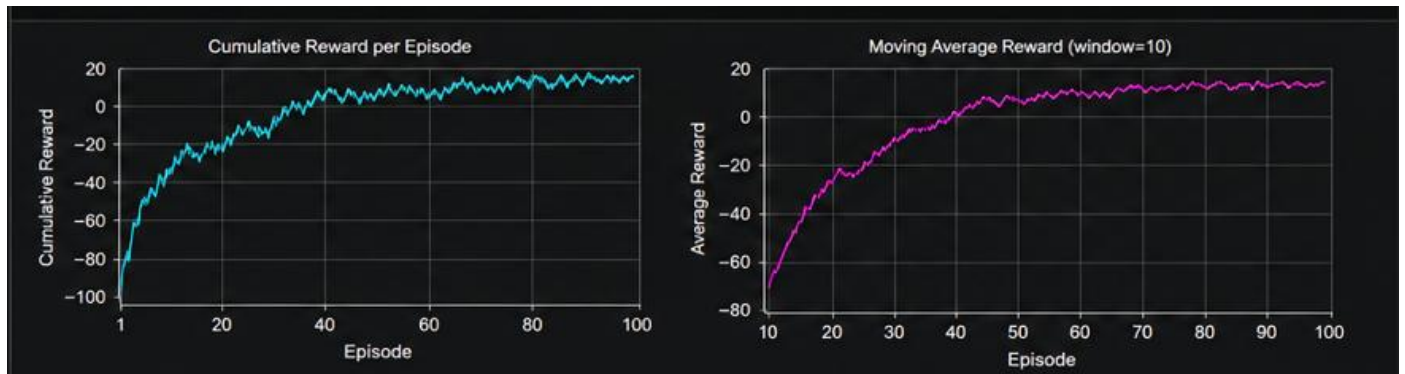
Q-VALUES AT EPISODE 100
-----
State 0 : [-0.64  0.58 -0.72  4.64]
State 10: [ 3.18  6.42  2.87  5.02]

FINAL Q-TABLE (rows = states 0..15, cols = [UP, DOWN, LEFT, RIGHT])
State |    UP    DOWN    LEFT    RIGHT
-----|-----
0      | -0.64  0.58  -0.72  4.64
1      | -0.72  0.76  -0.81  6.26
2      | -0.81  0.95  -0.90  7.96
3      | -0.90  1.18  -1.00  8.83
4      | -0.72  0.46  -0.81  3.31
5      | -0.81  0.64  -0.90  -5.00
6      |  0.00  0.00  0.00  0.00  <-- OBSTACLE
7      | -0.90  1.46  9.04  8.25
8      | -0.64  0.31  -0.72  2.41
9      | -0.72  0.58  -0.81  3.82
10     | 3.18  6.42  2.87  5.02
11     | 5.63  9.01  7.40  8.82
12     | -0.55  0.14  -0.59  1.73
13     | -0.64  0.31  -0.72  2.41
14     | -0.72  0.58  -0.81  3.82
15     |  0.00  0.00  0.00  0.00  <-- GOAL

FINAL LEARNED POLICY (best action in each state)
Legend:
↑ = Up
↓ = Down
← = Left
→ = Right
X = Obstacle
G = Goal

| → | → | → | ↓ |
| ↑ | → | X | ↓ |
| ↑ | ↑ | → | ↓ |
| ↑ | → | → | G |
```

Output:



Representative Final Policy

A possible learned policy is:

→ → ↓ ↓
↑ → X ↓
↑ ↑ → ↓
↑ → → G

The exact arrows can vary depending on the random learning process and hyperparameters.

A possible optimal route from the starting state is:

$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3$

↓
S7
↓
S11
↓
S15

Therefore, the agent learns a path that avoids the obstacle and eventually reaches the goal.

Cumulative Reward per Episode

The cumulative reward is the total reward received by the agent during each episode.

Python Code

```

def step(state, action):
    r, c = state_to_pos(state)
    nr, nc = r, c
    if action == UP: nr -= 1
    elif action == DOWN: nr += 1
    elif action == LEFT: nc -= 1
    elif action == RIGHT: nc += 1
    # boundary check
    if nr < 0 or nr >= ROWS or nc < 0 or nc >= COLS:
        return state, -1, False # hit wall
    next_state = pos_to_state(nr, nc)
    if next_state == GOAL:
        return next_state, 10, True # reached goal
    if next_state == OBSTACLE:
        return state, -5, False # hit obstacle
    return next_state, -1, False # normal step

# Store cumulative rewards
rewards_per_episode = []
snapshots = {}

# Q-Learning
for episode in range(1, episodes+1):
    state = START
    total_reward = 0
    for step_count in range(max_steps):
        # epsilon-greedy action
        if random.random() < epsilon:
            action = random.choice(actions)
        else:
            action = np.argmax(Q[state])
        next_state, reward, done = step(state, action)
        old_q = Q[state, action]
        best_next_q = np.max(Q[next_state])
        # Q-learning update
        Q[state, action] = old_q + alpha * (reward + gamma * best_next_q - old_q)
        state = next_state
        total_reward += reward
        if done:
            break
    # decay epsilon

```

The plot generally starts with unstable or negative rewards because the agent initially explores the environment randomly.

With more training:

Cumulative

Reward



The exact shape will vary because exploration is random.

Justification of Convergence Behaviour

During the first few episodes, the agent does not know which actions are beneficial. Therefore, it explores many possible movements, including actions that produce negative rewards.

As the Q-table is repeatedly updated, the agent learns which state-action combinations provide higher long-term rewards. The Q-values of useful actions gradually increase, while actions that lead toward boundaries, obstacles or unnecessary movements receive lower values.

The decreasing epsilon value causes the agent to gradually shift from exploration to exploitation.

Therefore, the cumulative reward generally improves over the episodes and eventually becomes more stable.

A practical convergence criterion can be stated as:

$$\max_{s,a} |Q_{new}(s, a) - Q_{old}(s, a)| < 0.01$$

for several consecutive episodes.

When the changes in Q-values become very small and the cumulative reward becomes relatively stable, the policy can be considered approximately converged.

Results and Discussion

Experiment 1 Result

The Decision Tree classifier successfully learned the relationship between Iris flower measurements and their species. The model achieved high classification accuracy on the test data. The confusion matrix showed that most samples were correctly classified, with only a small number of misclassifications between the more similar Versicolor and Virginica classes.

The root feature was selected because it provided the greatest reduction in impurity. This demonstrates how a Decision Tree automatically identifies the most useful features for classification.

Experiment 2 Result

The Q-Learning agent successfully learned a policy for navigating the 4×4 Grid World. Initially, all Q-values were zero because the agent had no knowledge of the environment. As the number of episodes increased, the Q-values were updated based on rewards and future expected rewards.

After 100 episodes, the agent learned preferred actions for most states and was able to identify a route toward the goal while avoiding the obstacle. The cumulative reward became more stable as the policy improved.

Final Conclusion

The two experiments demonstrate two important CO4 machine-learning concepts.

The **Decision Tree experiment** shows how supervised learning can be used for classification. The tree selects useful splitting features using measures such as the Gini Index and can then classify unseen observations. Performance can be evaluated using accuracy, confusion matrix and classification-report measures.

The **Q-Learning experiment** demonstrates reinforcement learning, where an agent learns through interaction with an environment. The Q-table initially contains zero values and gradually learns the expected reward associated with different state-action pairs. After sufficient episodes, the agent extracts an optimal or near-optimal policy.

Thus, the experiments demonstrate both **classification-based decision making and reinforcement-learning-based sequential decision making**, which are central concepts of CO4.